

Experiment No.8

1. Title: ATM Machine using User Defined Exception Handling

2. Problem Statement:

Develop a Java-based ATM Banking Simulation System that allows users to check account balance, deposit money, and withdraw money through a menu-driven interface. Implement user-defined exceptions such as `InvalidAmountException` and `InsufficientFundsException`, and use `try`, `catch`, and `finally` blocks to handle errors gracefully. The application should display appropriate error messages and continue execution until the user chooses to exit.

3. Objectives:

- To develop a menu-driven ATM Banking Simulation System using Java.
- To implement account operations such as balance inquiry, deposit, and withdrawal.
- To demonstrate the use of exception handling using `try`, `catch`, and `finally` blocks.
- To create and use user-defined exceptions for handling invalid amounts and insufficient funds.
- To design a robust application that continues execution even after exceptions occur.

4. Outcomes:

Students will be able to

- Apply Object-Oriented Programming concepts to develop a real-world application.
- Implement and handle user-defined exceptions in Java.
- Use exception handling mechanisms to improve program reliability and fault tolerance.
- Develop menu-driven applications with effective user interaction.
- Analyze and manage runtime errors to ensure smooth program execution.

5. Software/Hardware/Other Requirements:

- Any CPU with Pentium Processor / similar, 256 MB RAM/ more, 1GB HDD / more.
- Operating System – ubuntu/Fedora 64bit OS
- Software: JDK 17 or later, Code Editor

6. Theory:

A. User-Defined Exception Throwing

- A user-defined exception (or custom exception) is an exception created by the programmer to represent application-specific error conditions that are not covered by Java's built-in exceptions.
- User-defined exceptions are created by extending the `Exception` class (checked exception) or `RuntimeException` class (unchecked exception).

- **Why Use User-Defined Exceptions?**

- To provide meaningful error messages.
- To handle application-specific errors.
- To improve code readability and maintainability

```
// User-defined Exception
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {    super(message);    }
}

public class VotingEligibility {
    static void checkAge(int age) throws InvalidAgeException {
        if (age < 18)
            throw new InvalidAgeException("Age must be 18 or above to vote.");
        System.out.println("You are eligible to vote.");
    }
    public static void main(String[] args) {
        try {
            checkAge(16);
        }
    }
}
```

```

        catch (InvalidAgeException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}

```

Output: Exception: Age must be 18 or above to vote.

- **Steps to Create a User-Defined Exception**

- 1) Create a class that extends Exception.
- 2) Define a constructor that accepts an error message.
- 3) Use the throw keyword to generate the exception.
- 4) Use throws in the method declaration.
- 5) Handle the exception using a try-catch block.

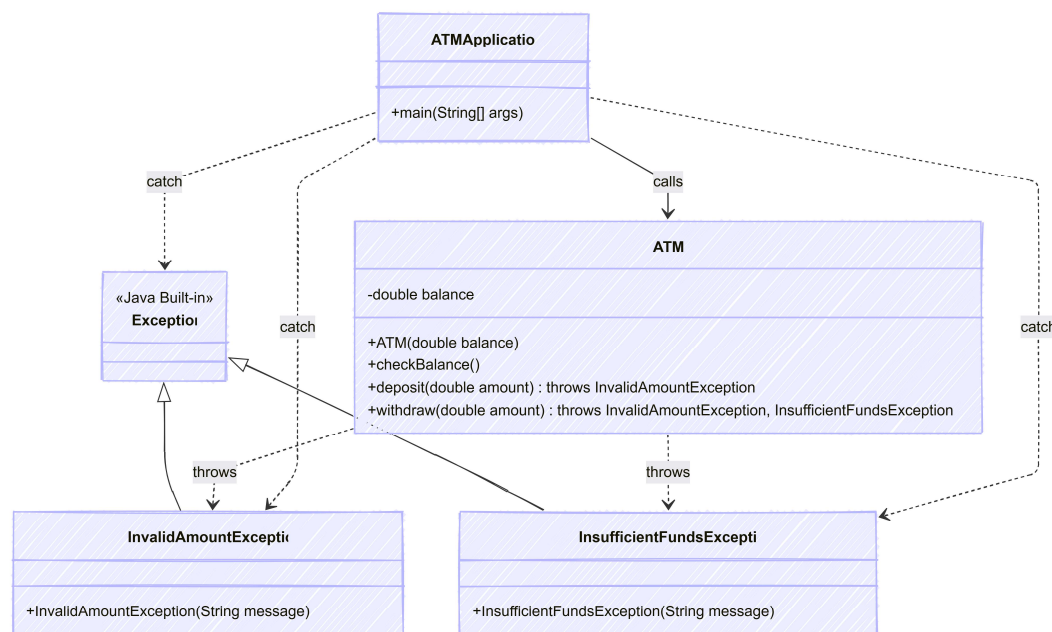
- **B. Short Description of the problem statement:**

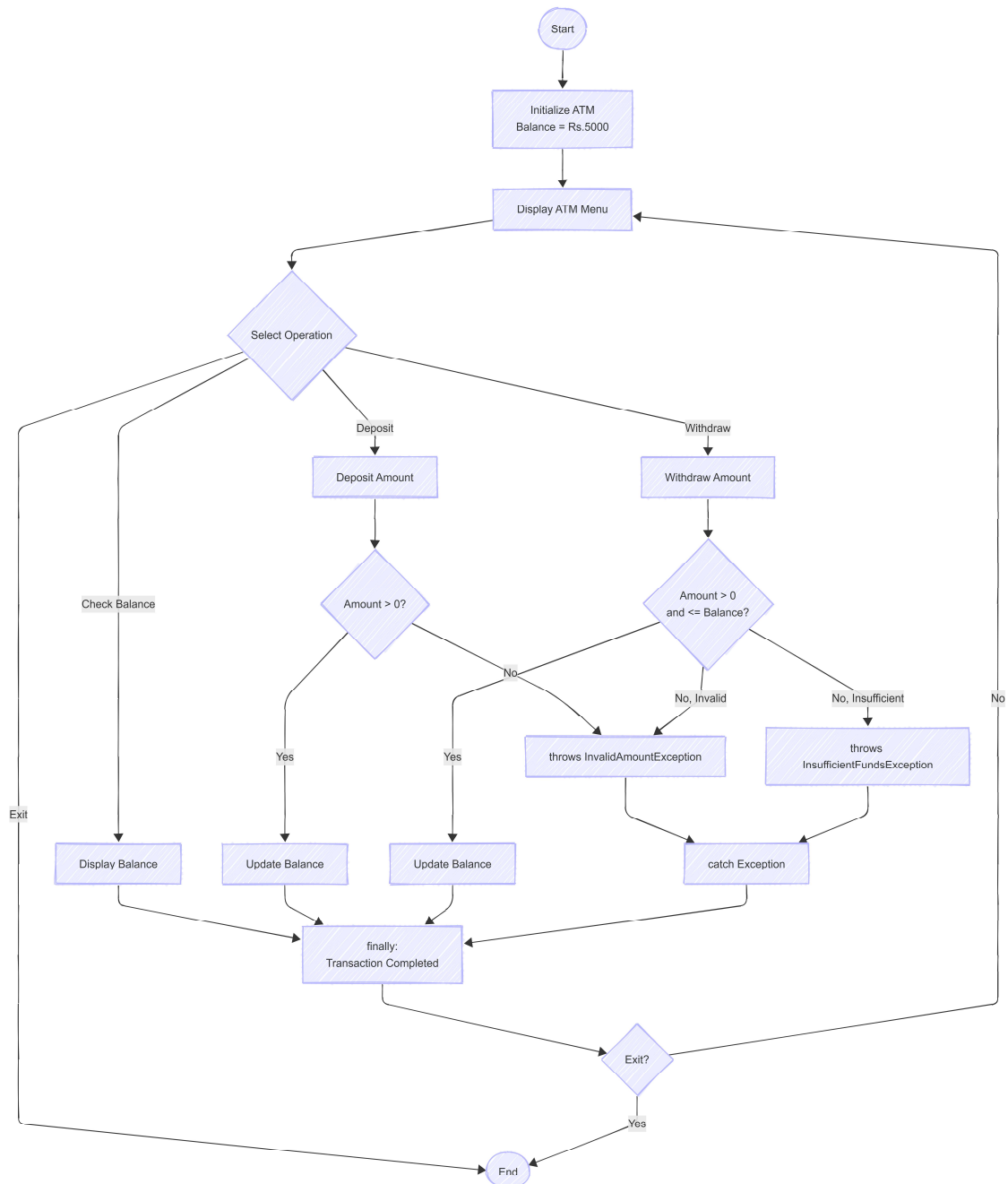
- The **ATM Banking Simulation System** is a Java-based console application that simulates the basic functionalities of an Automated Teller Machine (ATM).
- The application allows users to perform banking operations such as checking account balance, depositing money, and withdrawing money through a menu-driven interface.
- The primary objective of this project is to demonstrate the implementation of **Object-Oriented Programming (OOP)** concepts and **exception handling** mechanisms in Java.
- The system uses both built-in and user-defined exceptions to handle invalid transactions, insufficient account balance, and incorrect user inputs, ensuring that the application remains reliable and continues execution even when errors occur.

- **Key Features**

a. Check current account balance.	b. Deposit money into the account.
c. Withdraw money from the account.	d. User-defined exceptions: InvalidAmountException, InsufficientFundsException
e. Exception handling using try, catch, throw, throws, and finally.	f. Validation of user inputs and transaction amounts.
g. Displays meaningful error messages for invalid operations.	h. Ensures uninterrupted program execution after exception handling.
i. Demonstrates real-world banking transaction processing using Java	j. Menu-driven user interface for easy interaction.

7. Class Diagram:



Application Workflow Diagram:**8. Implementation:****Java Program for Wallet Application**

```

import java.util.Scanner;
// User-defined Exception for insufficient balance
class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) { super(message); }
}
// User-defined Exception for invalid amount
class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) { super(message); }
}
class ATM {
    private double balance;
    public ATM(double balance) { this.balance = balance; }
    public void checkBalance() {

```

```

        System.out.println("Available Balance: Rs." + balance);
    }
    public void deposit(double amount) throws InvalidAmountException {
        if (amount <= 0)
            throw new InvalidAmountException("Deposit amount must be greater
            than zero.");
        balance += amount;
        System.out.println("Rs." + amount + " deposited successfully.");
    }
    public void withdraw(double amount) throws InvalidAmountException,
        InsufficientFundsException {
        if (amount <= 0)
            throw new InvalidAmountException("Withdrawal amount must be
            greater than zero.");
        if (amount > balance) {
            throw new InsufficientFundsException("Insufficient balance for
            withdrawal.");
        }
        balance -= amount;
        System.out.println("Rs." + amount + " withdrawn successfully.");
    }
}

public class ATMApplication {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ATM atm = new ATM(5000);
        int choice = 0;
        while (choice != 4) {
            System.out.println("\n===== ATM MENU =====");
            System.out.println("1. Check Balance");
            System.out.println("2. Deposit Money");
            System.out.println("3. Withdraw Money");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");
            try {
                choice = sc.nextInt();
                switch (choice) {
                    case 1: atm.checkBalance();
                        break;
                    case 2: System.out.print("Enter deposit amount: Rs.");
                        double depositAmount = sc.nextDouble();
                        atm.deposit(depositAmount);
                        break;
                    case 3: System.out.print("Enter withdrawal amount: Rs.");
                        double withdrawAmount = sc.nextDouble();
                        atm.withdraw(withdrawAmount);
                        break;
                    case 4: System.out.println("Thank you for using ATM.");
                        break;
                    default: System.out.println("Invalid Menu Choice.");
                }
            }
            catch (InvalidAmountException e) {
                System.out.println("Invalid Amount Error: " + e.getMessage());
            }
            catch (InsufficientFundsException e) {
                System.out.println("Transaction Failed: " + e.getMessage());
            }
            catch (Exception e) {
                System.out.println("Invalid Input! Please enter numeric values
                only.");
                sc.nextLine(); // clear buffer
            }
            finally {
                System.out.println("Transaction completed.");
            }
        }
        sc.close();
    }
}

```

9. Conclusion:

The ATM Banking Simulation System successfully demonstrates the implementation of Object-Oriented Programming and user-defined exception handling in Java. The application ensures secure and reliable banking operations by effectively managing invalid transactions and runtime errors.

10. Review Questions & Exercises:**a. Fill in the Blanks**

Q. No.	Question	BT Level	CO	PO & PSO
i	A user-defined exception in Java is created by extending the _____ class.	BT ₂	CO ₂	PO ₁ , PSO ₁
ii	The keyword used in a method declaration to specify possible exceptions is _____.	BT ₁	CO ₂	PO ₁ , PSO ₁
iii	Multiple exceptions can be handled using multiple _____ blocks.	BT ₂	CO ₂	PO ₁ , PSO ₁

b. Answer the following:

Q. No.	Question	BT Level	CO	PO & PSO
i	Differentiate between throw and throws.	BT ₂	CO ₂	PO _{1,2} , PSO ₁
ii	Explain the difference between throw, throws, try, catch, and finally.	BT ₂	CO ₂	PO _{1,2} , PSO ₁
iii	Modify the ATM program so that a withdrawal is allowed only in multiples of Rs. 100. What exception would you create?	BT ₄	CO ₂	PO _{1,2} , PSO ₁
iv	Design a custom exception for an invalid ATM PIN and explain where it should be thrown and caught.	BT _{2,3}	CO ₂	PO _{1,2} , PSO ₁

c. Programming Assignments:

Que. No.	Question	BT Level	CO	PO & PSO
i	Mobile Recharge System: Develop a Java-based Mobile Recharge System that allows users to recharge prepaid mobile numbers, check balance, and view recharge history. Implement user-defined exceptions such as InvalidMobileNumberException and InvalidRechargeAmountException to handle invalid inputs and transactions.	BT ₃	CO ₂	PO _{1,3,5} , PSO ₁
ii	Gym Membership Management System: Develop a Java-based Gym Management System that allows members to register, renew memberships, and access facilities. Implement custom exceptions such as MembershipExpiredException and InvalidMembershipIDException.	BT ₃	CO ₂	PO _{1,3,5} , PSO ₁

11. References:

- E Balaguruswamy, (2023). Programming with Java: A Primer. 7th. India: McGraw Hill Education
- Herbert Schildt, (2021). Java: The complete reference, 13th edition. McGraw-Hill Education

Performance Indicators	
Process related (6 Marks)	
1	Logic formation (3)
2	Debugging ability (2)
3	Follow ethical practices (1)
Product related (4 Marks)	
4	Expected output (1)
5	Timely Submission (2)
6	Answer to sample questions (1)

Marks Obtained			Dated signature of Teacher
Process Related (6)	Product Related (4)	Total (10)	